

SynapseOS — Answers for the Team

Overview

This document answers the 53 questions Steve and Alex raised about SynapseOS, deduplicated (their lists overlap almost entirely) and grouped by theme rather than by who asked. Every answer is grounded in the actual proposal (research-methods/consolidated/SynapseOS_Proposal_Chapters_1 and the project’s design-decision log (docs/decisions.md, cited as D#) — nothing here is invented for the occasion. The short version, if there is time for only one paragraph: SynapseOS is an interface layer, not an operating system; its one required research contribution is an algorithm that figures out what a generated shell command will change *before* it runs, so a destructive mistake can be undone, and that algorithm is evaluated on its own, with no user study needed, against a held-out corpus of 776 commands. The 40-participant comparison against each person’s own OS is a second, independent piece of evidence, not the thing the thesis stands or falls on. Sections below: what SynapseOS is, safety and undo, why this counts as Computer Science, how it is evaluated, feasibility and architecture, and the literature review.

Table of Contents

- Overview
- 1. What SynapseOS actually is
- 2. Safety, undo, and what happens when things go wrong
- 3. Is this Computer Science, and where is the contribution
- 4. How the thesis is evaluated
- 5. Feasibility, timeline, and architecture
- 6. The literature review

1. What SynapseOS actually is

Is SynapseOS an operating system or an interface layer? An interface layer. It has no kernel — it runs on Debian’s. It occupies one slot in the Linux stack: the session/interface layer, the same slot a shell (bash) or a desktop environment (XFCE, GNOME) occupies. It calls the existing userland (coreutils, apt, etc.) unchanged. “OS” in the name is the product vision, not an engineering claim (docs/layers.md).

How is this different from ChatGPT / AI terminal tools? Three concrete differences, not a vibe: - **Runs fully local.** Inference is a small model (Qwen2.5-Coder-3B-Instruct, ~1.8 GB) served by Ollama on the same machine, CPU-only, no API key, no data leaving the device (D2). This matters specifically because SynapseOS sits at the OS level and can see every command and file; routing that through a cloud API would give an external provider continuous visibility into the user’s machine. - **It is not a chat window bolted onto a terminal — it is the session itself,** layered over an ordinary desktop the user keeps using (D27). Every command it proposes goes through the same reversibility check and confirmation gate, in every one of its three modes (CLI, TUI, GUI) — a generic “AI terminal” plugin doesn’t own that guarantee.

- **The recoverability algorithm.** Nothing in this comparison class computes, before a generated command runs, what pre-images are needed to undo it. Existing agentic CLI tools use pattern-matched permission lists, which can refuse a command but cannot restore what an approved one destroyed (see the abstract’s numbers under Q4 below).

Why Linux? Not a platform-superiority claim — a feasibility one. An agent layer that observes and acts on a whole desktop session needs a platform that permits that; Windows and macOS are proprietary and do not. Linux is where the experiment is *possible*, not what the experiment is *about* (D28). This is deliberately framed as a necessity rather than a novelty claim, because a novelty claim (“no one has done this on Linux”) expires the moment someone else ships one — necessity does not.

Why use XFCE and X11? XFCE is the lowest-effort, most standard traditional Linux desktop environment available — no custom kiosk compositor, no new Wayland layer-shell toolchain needs to be built for the thesis prototype, and it is a genuine, vendor-neutral desktop for the “compare against what the participant already uses” baseline design (D9). Its X11 session is used, not Wayland, because XFCE’s own Wayland session is still experimental as of version 4.20 — not something to gamble a 40-participant, one-shot study on (D12).

Why use Debian 13? Same reasoning as “why Linux,” one level down: Debian is the base because it’s what XFCE and the rest of the stack are built and packaged against on this project, and because a derivative-distribution path (the eventual “SynapseOS is its own distro” claim) runs through a Debian derivative, the same lineage as Ubuntu or Mint (docs/layers.md, “When It Becomes a Distribution”). It is adopted, not built (D1) — writing a kernel, init system, or package manager is explicitly out of scope.

2. Safety, undo, and what happens when things go wrong

How do the safety and undo mechanisms work? Two separate questions, two separate mechanisms (docs/safety-model.md): 1. **Is this command safe to run without asking?** A classifier judges every proposed command as Reversible (runs immediately) or Irreversible (blocks on an explicit y/N). As of the recoverability algorithm being on by default, this classification is backed by an *effect analysis*: instead of matching the command’s name against a list, it parses the command’s structure, works out what files it would read, create, write, remove, or change the metadata of, sees through wrappers (find -exec, xargs, loops, pipes, command substitution, sudo), and resolves targets that are only computed at run time (like rm \$(ls *.log)) using a read-only dry run in a sandbox. Anything it cannot resolve is treated as unrecoverable and asks — the analysis fails closed by design (D29, D34). 2. **If it runs, can it be undone?** Independently of the confirmation gate, an undo journal records what would be needed to reverse a command *before* it runs: a hardlink into trash for a pure deletion, a full content copy for something that overwrites a file in place, a metadata record for a permission change, a captured git commit SHA for git reset --hard, or an inverse apt/systemctl command for package and service state.

What can and cannot be undone? Can: anything with a bounded, nameable target — file deletions, in-place content overwrites, recursive permission/ownership changes, git reset --hard, and package/service installs, removals, and state changes (each inverted using the package manager’s own tools). Cannot, by design, not by oversight: a raw disk/partition write (dd/mkfs onto a block device — no filesystem-level target exists to protect), shred (its entire purpose is making content unrecoverable — backing it up first would defeat the point), eval and fetch-or-decode-then-execute (the effects are unknown until unreviewed code actually runs, so there’s nothing to capture in advance), and killing a process (process state, not file state — there’s nothing in the filesystem to restore). These five are documented as permanently out of scope, not “not built yet” (docs/safety-model.md).

What happens if the LLM generates a harmful or wrong command? It goes through the

same gate regardless of which model produced it. If the effect analysis judges it Irreversible or otherwise not provably safe, it stops and asks for a “yes” before running — and if the answer is no, nothing happens. If it runs (either because it was reversible, or because the user approved it), whatever can be captured already was, before execution, so it can be undone through the journal above. A wrong-but-harmless command (one that doesn’t accomplish the task but doesn’t destroy anything either) just fails and is reported as a failure — the loop reads the error and can try something else, up to a fixed step cap (D21).

How will you prevent users from blindly approving dangerous commands? Partially — this is treated as an open, honestly reported risk, not solved. The confirmation prompt always shows the actual command, states in plain language what kind of thing it is about to do, and — for a command the analysis has fully resolved — states plainly whether an undo will be possible afterward if things go wrong (see D30, and the live 2026-09 work fixing the approval wording so it doesn’t warn about deletion for a command that doesn’t delete anything). What it does *not* do is stop a user from approving something they don’t understand; see the next three questions.

What is the difference between confirmation and real safety? Confirmation is consent, not correctness — it means the user was shown the command and chose to proceed, not that the outcome will be what they wanted or that they understood the syntax. “Real safety” here means the recovery journal: even an approved command that turns out to be a mistake can, for the shapes above, be undone afterward. The thesis is explicit that these are different guarantees and does not conflate “the model asked for confirmation” with “I can undo it” (docs/safety-model.md, opening paragraph).

What is your fallback if the recoverability algorithm fails? It fails closed to the list-based classifier it was built to improve on, not to “nothing.” If the effects analysis cannot resolve a command’s targets — the sandbox isn’t available, the command isn’t one it models, resolution times out — the command is treated as unrecoverable and unknown, which means it asks for confirmation and captures nothing, exactly like the pre-existing pattern-list classifier’s behavior. That classifier is always still consulted underneath the analysis and can only add a confirmation, never remove one (docs/safety-model.md, “The list stays the baseline and the fallback”).

Can a novice user understand what they are approving? Does showing the command help or confuse them? If they can’t read Bash, how meaningful is their confirmation? These are genuinely open questions the study is designed to surface, not ones the design claims to have already answered. What is known going in: the literature this project builds on (VoicePilot’s study with motor-impaired participants) found that a preview-and-confirm step is a *necessary* design element even for users who can’t verify the technical content directly — it gives them a stopping point, not necessarily technical comprehension. SynapseOS’s gate shows the raw command (for anyone who can read it) *and* a plain-language reason. Whether that combination is legible to a non-technical participant, whether they over-trust it, and whether showing raw Bash actually helps or just adds noise for that population are exactly what the qualitative interview and the “trust in the system” question in the post-study interview are for (see Q22 below) — this is measured, not assumed.

How will you test the undo function without risking real data? The recoverability algorithm’s evaluation never touches a person’s real data at all: it runs against a held-out corpus of sampled commands inside a sandbox (bubblewrap), with ground truth established by actually executing each command in that sandbox and diffing the filesystem before/after — never a human label, never a live machine. Undo mechanisms are covered by their own automated tests that create disposable scratch files and directories, run the command, invoke undo, and assert the result. The user study is a separate matter: participants work through predefined tasks on evaluation-lab machines with data set up for the study, not their own files.

3. Is this Computer Science, and where is the contribution

Is this thesis too ambitious? It was, and that’s a documented, resolved concern rather than a live one. An adviser scope-management comment prompted an explicit decision (D36, 2026-09-26): the recoverability algorithm and a working, demoable prototype are the mandatory deliverable; the 40-participant comparative user study is a secondary, “nice to have” piece of evidence, evaluated independently. If the study can’t be fully run, the algorithm’s evaluation stands on its own with no participants required (D32).

Are you sure this is Computer Science and not just an application project? What is the algorithmic part of the work? The algorithm is: *given an arbitrary shell command produced by a language model, find what it would change before it runs, and where recovery needs preparation, compute the minimal set of pre-images sufficient to restore the prior state*. Concretely, it has two abilities: (1) **composition through wrappers** — analyzing a generated line as the commands that actually mutate something, seen through `find -exec`, `xargs`, loops, substitutions, pipelines, `sh -c`, `sudo`; and (2) **resolution of run-time targets** — where a target is only computed while the command runs (`rm $(ls *.log)`, `find -delete`), the analysis learns the concrete targets with a read-only dry run before the real command executes, so a capture plan can cover exactly the right things. Its formal basis is a stated effect semantics: a command is represented as the set of filesystem effects its syntax implies (read/create/write/remove/metadata-change), resolved against a working directory and composed through the shell operators that join commands into one line (`algorithms.md`; D29, D34). This is distinguished explicitly from the permission-list approach every comparable agentic tool uses, which can refuse a command by matching its name but takes no pre-images and so cannot restore what an approved command destroyed.

If the LLM is doing the translation, where is your group’s contribution? The translation step (natural language → a candidate command) is explicitly *not* claimed as the contribution — it’s a stock instruction-tuned model, and the paper says so plainly. The contribution starts *after* the model has produced a command: deciding what it would actually do, and computing how to undo it, is pure static/dynamic program analysis over the command’s syntax and the filesystem’s current state, and it runs the same regardless of which model or which UI produced the string. If the conversational interface were removed entirely, the algorithm would still be a real contribution on its own, evaluated on its own corpus with no participants (D32) — that’s the specific test an adviser asked, and the answer is yes.

How can you prove SynapseOS is better than the existing desktop? What if it performs worse — is the thesis still successful? “Better than the existing desktop” is not what’s being tested; a within-subjects comparison against each participant’s *own* operating system is (see Q4 below), and the two contributions are evaluated independently (D32). The algorithm’s result does not depend on the user study at all — it’s a corpus study against a fixed evaluation set. So a weak or null result in the user study does not invalidate the algorithm’s contribution, and a weak result in the user study is itself a reportable, honest finding (this is explicitly the point of D36 and D32 — protecting one contribution from depending on the other).

How will you separate model failure from interface failure? Structurally, by what each layer can independently be blamed for. Telemetry records, per step: what command was proposed, its classification, whether it was gated, whether it ran, its exit code and output, and separately, whether the natural-language answer given to the user matched what the commands actually showed. A wrong or nonsensical *command* is a model/translation failure; a *correct* command that the interface then mishandles, gates incorrectly, or fails to undo is an interface failure. This separation is actively being used already in day-to-day development — e.g., a bug where the answer stated a fact no command had actually checked was diagnosed and fixed as an interface-level grounding problem, independent of which model was running (recorded in `docs/open-problems.md` and `docs/decisions.md` D40).

What is the biggest technical risk of this project? What is the biggest methodological risk? Technical: the model itself is a small (3B-parameter), CPU-only local model, and its mistakes are numerous and specific — the project has already hit and fixed several live (a model copying a shell prompt into its own answer, mislabeling one command’s output as answering two different questions, a working-directory prompt change that dropped a test-suite score from 100% to 58% overnight). The project’s response, decided explicitly (D40), is to push as many of these fixes into code (schema-constrained model output, output verification) rather than hand-tuned prompt wording, specifically because prompt-level fixes don’t generalize across models and can regress silently. Methodological: the algorithm’s strongest evidence so far is a pilot (one annotator, a small sample, fixtures written after seeing the results) — a held-out, pre-registered round is what would actually carry weight, and is explicitly flagged as pending rather than claimed as done (D34).

What is the minimum viable version of SynapseOS that is still acceptable for thesis defense? Per D36: the recoverability algorithm, evaluated on its corpus, plus a working prototype that can be demoed live (TUI mode is sufficient — it doesn’t require the GUI/XFCE session work to be finished). The 40-participant study, and GUI mode specifically, are the parts explicitly allowed to slip without invalidating the thesis.

4. How the thesis is evaluated

How will you evaluate if SynapseOS is better? Two independent evaluations, because they’re two different kinds of claim (D32): - **The algorithm (RQ1):** measured against a held-out corpus of 776 shell commands, with ground truth from actually executing each one in a sandbox and diffing the filesystem — never a human label. Result so far: a pattern-matching list lets 41% of commands that destroy something run silently; the recoverability algorithm lets 0.2% run silently and additionally captures and restores 88% of the losses that can be executed; a list flipped to refuse everything it doesn’t recognize is equally safe (near-0% silent loss) but restores nothing, because it never takes a pre-image. This comparison needs no participants. - **The interface (RQ2-RQ4):** a within-subjects study, 40 participants (20 novice, 20 command-line fluent), each completing matched tasks under SynapseOS and under their own OS, with task completion time, error rate, System Usability Scale, NASA-TLX workload, and a semi-structured interview on trust and ease of use (D5, D6).

Why compare against each participant’s own OS? Because a single fixed Linux/GNOME baseline would confound the result with OS unfamiliarity: most participants have never used a Linux desktop, so a bad performance there would reflect not knowing Linux, not the interface being tested. Comparing against whatever OS the participant already uses daily (Windows, macOS, or Linux) is a harder, more ecologically valid opponent — a positive result against someone’s home environment is a stronger claim than beating an OS nobody in the sample actually uses (D9).

How will you separate model failure from interface failure? — answered in section 3, Q16.

Can the system handle ambiguous instructions? Partially, and this is measured rather than assumed. The system keeps session-scoped conversational memory so a follow-up like “move it to Downloads” can resolve “it” against what was just discussed (D10) — see Q53 below. Genuinely ambiguous or out-of-scope phrasing is not resolved by asking a clarifying question at this stage; it’s handled by the same mechanism as an out-of-scope request (next answer). Documented, unfixed limitation: ambiguity in user instructions is a known limitation of NL-to-shell translation broadly, and the project’s own live testing has found and logged specific ambiguity failures (e.g., an answer conflating two different quantities from one command’s output) as open items.

How will the system respond when the user asks something outside its scope? It says

so, plainly, rather than guessing or hallucinating an action. If the model decides a request has no shell-command equivalent, it responds explaining what it can and cannot do (files, processes, packages, text, network settings — but nothing with no command-line equivalent, like clicking a button in a GUI application or editing an image) and suggests how to phrase something it can act on. This is a deterministic response path, not model improvisation — a small set of conversational cases (greetings, “what can you do,” thanks) are answered locally without invoking the model at all, specifically so a polite “hi” is never misread as an unsupported task (D31).

Will users trust the system too much? How will you measure that? This is an explicit, named risk the study is built to surface, not a design that’s assumed safe. It’s measured via the post-study semi-structured interview, which directly asks about trust in the system, alongside the objective telemetry — confirmation prompts triggered, how often they were approved vs. declined, and how often (if a fallback exists) participants used it — cross-referenced against task outcomes to see whether approvals tracked genuine understanding or were rubber-stamped.

5. Feasibility, timeline, and architecture

Can this be realistically implemented within the thesis timeline? Can your group realistically build this? (*asked twice, same answer*) Yes, specifically because of D36’s rescoping: the mandatory deliverable is the algorithm plus a demoable prototype, not the full 40-participant study. The core execution loop, the classifier, the effect analysis, the undo journal, and all three interface modes (CLI, TUI, GUI) are implemented and tested now, ahead of the user study.

Which parts are already working and which parts are still planned? Working now: the propose → classify → confirm → execute loop; the effect-analysis recoverability algorithm (on by default); all five undo mechanisms; CLI, TUI, and GUI-session-launch modes; session-scoped conversational memory; the corpus/oracle pipeline used for the algorithm’s own evaluation. Still open/planned: the held-out, pre-registered evaluation round for the algorithm (the current strongest result is a pilot); the 40-participant comparative study itself (recruitment, IRB, execution); a few specific undo/classification edge cases logged as known gaps (`docs/open-problems.md`); and the fully packaged, bootable distro image (the current prototype is an application installed on Debian, not yet an installable ISO).

What happens if the GUI mode is not ready before user testing? This is planned for, not improvised. GUI mode is architecturally just the same TUI program launched fullscreen inside a standard XFCE login session (D27) — it is not a separately built graphical application, so there’s comparatively little left to go wrong independently of TUI mode, which is already working. There is also a participant-accessible fallback back to the underlying XFCE desktop if the interface becomes unresponsive mid-task; every use of it is logged, and any task where it was used is excluded from the primary comparison and reported separately as its own metric, rather than silently counted as a SynapseOS success or failure (D20).

Why use XFCE and X11? Why use Debian 13? — answered in section 1.

What are the minimum hardware requirements? Can Qwen2.5-Coder-3B-Instruct run fast enough on CPU-only hardware? Documented minimums: a 64-bit x86 processor (2 cores minimum, no GPU required), 4 GB RAM for CLI/TUI mode (8 GB recommended for GUI mode), 10 GB storage, and the ~1.8 GB quantized model running CPU-only through Ollama. It runs, but “fast enough” is condition-dependent: on the reference development machine (an aging 2-core/7.7 GB laptop), a warmed-up model answers in a few seconds, but under real memory pressure (browser, IDE, and the model all resident at once, pushing the system into swap) the same request can take 20–40+ seconds — a measured, documented effect, not a guess. The study’s dedicated evaluation machine is provisioned to the 8 GB

GUI-mode recommendation specifically to avoid this.

How will you fine-tune or configure the model? As of the current build, it is not fine-tuned — the thesis deliberately evaluates an interface built on an existing, off-the-shelf model rather than training one, because training a model is a different study (Table 1.1 of the proposal). Configuration happens at the request level: model options (temperature, context window) are set explicitly rather than left to defaults — a real bug was found and fixed where Ollama’s default context window silently truncated prompts — and, more recently, the model’s *reply shape* is constrained in code (JSON-schema-constrained output for the command-proposal step) rather than steered through prompt wording, specifically so fixes generalize across models instead of being tuned to this one’s quirks (D40). LoRA fine-tuning on an NL2Bash-style corpus was considered as a stretch goal but is not committed scope.

What if the model accuracy is too low? This is treated as a real, reportable possibility, not assumed away — the whole RQ1/RQ2 split (D32) exists partly so a weak translation-model result doesn’t invalidate the algorithm’s independent evaluation. Practically, low accuracy shows up as more UNSUPPORTED/step-limit outcomes and more corrected retries within the bounded loop (max 5 steps, D21) rather than as silent wrong actions, because every step — however the command was produced — still goes through the same classifier and confirmation gate.

What if participant tasks require GUI-only interaction outside shell expressibility? The system says so rather than attempting a workaround — anything with no command-line equivalent (clicking a button in a graphical application, editing an image) is explicitly out of scope and reported as such (D7). The study’s task suite is designed around shell-expressible tasks for exactly this reason (file management, process control, package management, text processing, network configuration), and boundary tasks that fall outside that are a deliberate part of the task suite (see the appendix’s “Boundary tasks — outside shell expressibility” table) to measure how the system handles that limit, not to pretend it doesn’t exist.

Does SynapseOS run as a standalone window manager or a desktop panel plugin? Neither. It does not manage windows or replace XFCE’s own window manager, panel, or session manager — none of that is touched. In GUI mode it is a single fullscreen application window (an existing terminal emulator) running on top of an ordinary, otherwise-unmodified XFCE session that keeps running underneath it (D12, D27).

How does the session layer intercept desktop inputs globally? It doesn’t intercept anything globally — there is no keyboard/input hooking or global capture. It is a normal foreground application window that has keyboard focus while it is fullscreen, exactly like any other program the desktop launches; the participant’s ability to switch away (the D20 fallback) exists precisely because SynapseOS does not own the input stack.

What happens to standard GUI applications when SynapseOS is active? Nothing — they keep running exactly as before. XFCE and everything underneath it (window manager, file manager, browser, any other open application) stays up and usable the entire time; SynapseOS is a window layered on top of that session, not a replacement for the session (D27, docs/layers.md).

How do you handle multi-turn context when a user references “that file from earlier”? Session-scoped conversational memory: a rolling window of the conversation (with old turns dropped and verbose command output compressed once the context budget fills up) is kept for the life of the login session and passed back to the model on each new request, so a pronoun like “it” or “that file” can resolve against what was just discussed (D10). It is cleared on logout/reboot — nothing persists across sessions in the current design, which was a deliberate simplicity/privacy trade-off, not an oversight.

6. The literature review

Why did you choose Berkeley Unix Consultant as a related study? It’s the earliest serious system in this exact tradition — an operational, late-1980s natural-language interface that let non-expert users learn about and interact with Unix through conversation, built on formal knowledge representation. It’s included specifically because its authors identified *knowledge acquisition* (hand-engineering domain knowledge) as the bottleneck that limited it — a concern that directly foreshadows the modern problem of LLM agents needing per-application grounding data. It is also, four decades later, still the system that comes closest in ambition to a full conversational session layer, which makes it the natural historical anchor for the gap this thesis addresses.

How is NL2Bash related to SynapseOS? NL2Bash-line work (Westenfelder et al.) is the closest existing evidence on the exact sub-problem SynapseOS’s translation step depends on: LLMs converting natural language into Bash commands with usable accuracy, including model selection at small parameter counts — it’s part of why Qwen2.5-Coder-3B-Instruct was chosen (D3). Its authors also name two open limitations — ambiguity in instructions and lack of system context (current directory, history, environment) at translation time — that SynapseOS’s persistent conversational session is specifically positioned to help with, since it can supply exactly that context.

How is NaSh related, and what does SynapseOS add beyond it? NaSh (Gyawali et al.) is the closest related work on the safety side: it proposes a guardrail layer between a user’s natural-language input and an LLM-generated shell command, evaluating the candidate against safety/reversibility/confirmation criteria and giving the user a structured preview before execution — the same shape as SynapseOS’s confirmation gate. It is a *design proposal*, published without an implementation or an empirical evaluation, and its authors explicitly leave the design of recovery/rollback mechanisms as open future work. SynapseOS adds exactly that: a working, implemented, and evaluated recovery mechanism (the recoverability algorithm and undo journal) where NaSh has none, plus a built and evaluated system rather than a proposal.

Why include VoicePilot if it is about assistive robots, not desktops? Not for its subject matter (physically assistive robotics) but for its method: it is the most rigorous human-centered evaluation in the entire LLM-speech-interface literature, and it derives concrete, evidence-based design guidelines from a real study with motor-impaired participants — among them that preview-and-confirm workflows are a *necessity*, not an optional nicety. That guideline directly informed the reasoning behind SynapseOS’s own confirmation-gate design (D30). It is cited for its methodological rigor and its transferable design finding, not as a claim that SynapseOS targets the same population.

Why include OSWorld if your study is about a human-operated conversational interface? As the evaluation-environment benchmark for the *other* branch of this field (autonomous computer-using agents), used as a contrast rather than a direct comparison. It establishes just how far autonomous GUI agents currently are from reliable long-horizon task completion (the best frontier system completes only 20.6% of OSWorld 2.0’s long-horizon tasks outright) — which is direct supporting evidence for D21’s decision to build SynapseOS as a bounded, human-supervised, confirmation-gated loop rather than a fully autonomous agent. It also matters because OSWorld includes Ubuntu only as one of several evaluation platforms, not as a deployment target — which is part of the literature-review finding that no surveyed system treats the Linux desktop as a primary target the way this thesis does.

What is the difference between GUI agents and shell-based conversational interfaces? GUI (computer-using) agents act by combining native application APIs with vision-based grounding — recognizing buttons, icons, and layout on screen and simulating clicks — to operate existing graphical applications. A shell-based conversational interface, SynapseOS’s approach, translates intent directly into commands run through the existing

command-line toolchain; there’s no visual perception step and no simulated input. The trade-off is direct: GUI agents can in principle reach anything visible on screen but inherit visual grounding as an unsolved bottleneck and are expensive and platform-fragile; shell-based agents are cheaper, more reliable within their scope, and — per a cited finding that verification is what makes CLI agents outperform GUI agents — easier to check before acting, but are bounded to whatever the command line can express (D7).

Why not build a vision-based GUI agent instead of a Bash-based one? Rejected explicitly and for tractability, not because it’s a worse idea in principle (D7). AT-SPI accessibility-tree automation and vision-based simulation both require deep, application-specific integration that varies across GUI toolkits and breaks with application updates — too fragile to build reliably and evaluate in a controlled study within a thesis timeline. Constraining to CLI produces a scope that is actually buildable and gives a clean, verifiable claim, while the conversational-interface value proposition (act without knowing exact syntax) holds fully within that scope.

What is the relevance of accessibility studies if users with motor accessibility needs are excluded from the initial evaluation? Honestly: relevant for their *design guidance*, not because the current 40-participant study specifically recruits or accommodates that population — it doesn’t, and that is a real, named limitation rather than a hidden one. The accessibility/speech-driven-interaction literature (VoicePilot, DexAssist, and related work) provides the most rigorous human-centered evaluations in the whole field and derives guidelines — preview-and-confirm, execution speed, fallback strategies — that generalize to any conversational interface acting on a real system, which is why SynapseOS’s own confirmation design draws on them. The literature review itself names this as a structural asymmetry in the field: accessibility work evaluates specific populations rigorously but narrowly, while the mainstream agent literature (including this thesis’s own study population — novice and command-line-fluent general users) evaluates broader populations but without that same rigor on trust and safety. That asymmetry is acknowledged, not resolved, by this thesis.

What is the strongest existing work closest to SynapseOS? No single system is closest on every axis, so the honest answer names the closest on each: **in ambition**, the Berkeley Unix Consultant (1988) — full conversational consultation about Unix, four decades before LLMs made it broadly tractable, but pre-LLM and never scaled past demonstration. **In safety design**, NaSh — the same guardrail/preview/confirm shape, but unimplemented and unevaluated. **In operational maturity**, UFO² — the most operationally advanced desktop-agent system surveyed, but Windows-only, built as an automation layer deliberately confined to a separate desktop from the user (the opposite of SynapseOS’s “layered over the same session” design), and not evaluated against the user’s own conventional workflow. **In stated concept**, AgentOS (2026) — proposes a natural-language portal in place of the graphical desktop, closest in spirit to SynapseOS’s framing, but offered as a research direction with no built system or evaluation. The gap all four leave, and the one this thesis fills, is: no surveyed system is an implemented and empirically evaluated conversational session layer for the Linux desktop, controlled against the conventional workflow it would supplement.

What is the main limitation of your literature review? The review’s own stated criteria admit five kinds of work (NL-to-action translation, GUI/web/mobile agents, accessibility/speech interaction, evaluation environments, and controlled human-vs-agent comparisons) — which is broad enough to be fair, but it also means the review spans several distinct sub-fields that don’t otherwise cite each other much, so the “gap” claim rests on a synthesis across fields rather than a single well-established prior debate. Separately and more concretely: the strongest existing counter-argument against the whole premise (Shneiderman’s direct-manipulation critique, and later empirical work showing a persistent gap between what users expect a conversational agent to understand and what it delivers) is taken seriously and two of its three concerns are addressed by design (showing the actual command and output rather than a prose summary; the confirmation gate and undo journal

restoring the reversibility direct manipulation gets for free) — but the third concern, that a user cannot see the boundary of what the system can do, is explicitly *not* solved by the current design and is carried forward as an acknowledged, open limitation rather than claimed away.